Testing fundamentals

21A. Testing Fundamentals

Questions: 4 **Difficulty:** Mid **Companies:** Google, Amazon, Flipkart, Razorpay, Swiggy, Microsoft, Atlassian

Q 21.1 What's the testing philosophy for frontend? Explain the testing trophy and what to test.

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart, Atlassian

The testing trophy (coined by Kent C. Dodds, creator of React Testing Library) replaces the traditional testing pyramid for frontend:

JAVASCRIPT Copy

```
┌─────────┐
│   E2E   │          Few - expensive, slow, but high confidence
┌┴─────────┴┐
│Integration│        MOST tests should be here
┌┴───────────┴┐
│    Unit     │      Some - fast, but test in isolation
┌┴─────────────┴┐
│ Static Analysis│   ESLint, TypeScript - catches bugs before test
└───────────────┘
```

clicks through the entire checkout flow is expensive and flaky. An integration test that renders a component, simulates user interaction, and verifies the output — that's the sweet spot.

What to test — behavior, not implementation:

The user doesn't care that your state variable changed from `false` to `true`. They care that clicking the button shows the dropdown.

JSX Copy

```jsx
// BAD: Testing implementation details
test('sets isOpen to true when clicked', () => {
  const { result } = renderHook(() => useDropdown());
  act(() => result.current.toggle());
  expect(result.current.isOpen).toBe(true);
  // Breaks if you rename the variable or restructure the hook
});

// GOOD: Testing user-visible behavior
test('shows dropdown options when trigger is clicked', () => {
  render(<Dropdown options={['React', 'Vue', 'Svelte']} />);
  fireEvent.click(screen.getByRole('button', { name: /select/i }));
  expect(screen.getByRole('listbox')).toBeVisible();
  expect(screen.getByRole('option', { name: 'React' })).toBeInTheDocument(
});
```

What NOT to test:

- Third-party library internals (don't test that React renders correctly)
- Exact CSS styles (fragile, changes constantly — use visual regression instead)
- Implementation details (state values, function calls, internal methods)
- Trivial code (a component that renders a string with no logic)

What TO test:

- User interactions (click, type, submit, keyboard navigation)

- Conditional rendering (error states, loading states, empty states)
- Data transformations (formatting, filtering, sorting)
- Edge cases (empty inputs, very long strings, special characters)
- Accessibility (keyboard navigation, ARIA attributes, focus management)

Follow-up: "What coverage target do you aim for?"

Coverage numbers are misleading. 100% coverage doesn't mean 100% tested — you can cover every line without testing meaningful behavior. A pragmatic target: 80% line coverage as a minimum gate, with emphasis on critical paths (checkout, authentication, payment) having near-100% coverage. Never chase coverage numbers for their own sake.

Q 21.2 How do you write unit tests? Explain Jest/Vitest patterns.

Difficulty: Mid **Asked at:** Every company

Unit tests verify individual functions and modules in isolation. For frontend, this typically means utility functions, data transformations, custom hooks (without rendering), and business logic.

Jest and Vitest have nearly identical APIs. Vitest is faster (native ESM, Vite-powered) and is the default for new projects. Jest still dominates in existing codebases.

JAVASCRIPT Copy

```
// utils/formatPrice.js
export function formatPrice(cents, currency = 'INR') {
  if (typeof cents !== 'number' || cents < 0) return '—';
  const amount = cents / 100;
  return new Intl.NumberFormat('en-IN', {
    style: 'currency',
    currency,
```

JAVASCRIPT Copy

```javascript
// utils/formatPrice.test.js
import { describe, it, expect } from 'vitest';
import { formatPrice } from './formatPrice';

describe('formatPrice', () => {
  it('formats cents to INR currency string', () => {
    expect(formatPrice(129900)).toBe('₹1,299.00');
  });

  it('handles zero', () => {
    expect(formatPrice(0)).toBe('₹0.00');
  });

  it('handles different currencies', () => {
    expect(formatPrice(9999, 'USD')).toBe('$99.99');
  });

  it('returns dash for negative values', () => {
    expect(formatPrice(-100)).toBe('-');
  });

  it('returns dash for non-number input', () => {
    expect(formatPrice('abc')).toBe('-');
    expect(formatPrice(null)).toBe('-');
    expect(formatPrice(undefined)).toBe('-');
  });
});
```

Mocking — isolating the unit under test:

JAVASCRIPT Copy

```javascript
// orderService.js
import { fetchUser } from './userApi';
import { calculateTax } from './taxCalculator';

export async function createOrderSummary(userId, items) {
```

```
  return { user: user.name, subtotal, tax, total: subtotal + tax };
}
```

JAVASCRIPT Copy

```
// orderService.test.js
import { vi, describe, it, expect } from 'vitest';
import { createOrderSummary } from './orderService';

vi.mock('./userApi', () => ({
  fetchUser: vi.fn().mockResolvedValue({ name: 'Ankit', state: 'KA' }),
}));

vi.mock('./taxCalculator', () => ({
  calculateTax: vi.fn().mockReturnValue(180),
}));

describe('createOrderSummary', () => {
  it('assembles order summary correctly', async () => {
    const items = [{ price: 500 }, { price: 300 }];
    const summary = await createOrderSummary('user-1', items);

    expect(summary).toEqual({
      user: 'Ankit',
      subtotal: 800,
      tax: 180,
      total: 980,
    });
  });
});
```

When to mock vs when not to:

Mock: external APIs, timers, browser APIs (localStorage, navigator), expensive computations you've already tested separately.

Don't mock: the thing you're testing, simple utilities, data transformations. If you mock everything, your test only proves that your mocks work.

Q 21.3 How do you test React components with React Testing Library?

Difficulty: Mid **Asked at:** Every company

React Testing Library (RTL) encourages testing components the way users interact with them — by finding elements through their accessible roles, labels, and text, not through CSS selectors or component internals.

Testing a search form:

JSX Copy

```jsx
// SearchForm.jsx
function SearchForm({ onSearch }) {
  const [query, setQuery] = useState('');

  function handleSubmit(e) {
    e.preventDefault();
    if (query.trim()) onSearch(query.trim());
  }

  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="search">Search products</label>
      <input
        id="search"
        type="text"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Type to search..."
      />
      <button type="submit">Search</button>
    </form>
  );
}
```

JAVASCRIPT Copy

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { vi, describe, it, expect } from 'vitest';
import SearchForm from './SearchForm';

describe('SearchForm', () => {
  it('calls onSearch with the query when submitted', async () => {
    const user = userEvent.setup();
    const onSearch = vi.fn();

    render(<SearchForm onSearch={onSearch} />);

    const input = screen.getByRole('textbox', { name: /search products/i }
    await user.type(input, 'wireless headphones');
    await user.click(screen.getByRole('button', { name: /search/i }));

    expect(onSearch).toHaveBeenCalledWith('wireless headphones');
  });

  it('does not call onSearch when query is empty', async () => {
    const user = userEvent.setup();
    const onSearch = vi.fn();

    render(<SearchForm onSearch={onSearch} />);
    await user.click(screen.getByRole('button', { name: /search/i }));

    expect(onSearch).not.toHaveBeenCalled();
  });

  it('trims whitespace from query', async () => {
    const user = userEvent.setup();
    const onSearch = vi.fn();

    render(<SearchForm onSearch={onSearch} />);

    await user.type(screen.getByRole('textbox', { name: /search products/i
    await user.click(screen.getByRole('button', { name: /search/i }));

    expect(onSearch).toHaveBeenCalledWith('shoes');
  });
});
```

RTL query priority (use in this order):

1. `getByRole` — accessible role + name (`getByRole('button', { name: /submit/i })`)
2. `getByLabelText` — form elements by their label
3. `getByPlaceholderText` — input by placeholder (less robust than label)
4. `getByText` — non-interactive elements by text content
5. `getByTestId` — last resort, escape hatch (`data-testid="custom-widget"`)

Prefer `getByRole` because it tests accessibility simultaneously — if `getByRole('button')` can't find your element, it means screen readers can't find it either.

`userEvent` **vs** `fireEvent`:

`userEvent` simulates real user interactions (typing triggers focus, keydown, input, keyup events in sequence). `fireEvent` dispatches a single synthetic event. Always use `userEvent` for realistic tests — it catches bugs that `fireEvent` misses (like a keydown handler that blocks typing).

Follow-up: "How do you test a component that fetches data?"

See Q21.4 — use MSW to mock the API at the network level.

Q 21.4 How do you write integration tests? Explain API mocking with MSW.

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart, Razorpay

Integration tests render a component (or a group of components) with their real dependencies — hooks, context, routing — and mock only the external boundary (API calls). MSW (Mock Service Worker) intercepts network requests at the network level, so your components use their real fetch/axios code.

MSW setup:

```
// mocks/handlers.js
import { http, HttpResponse } from 'msw';

export const handlers = [
  http.get('/api/products', ({ request }) => {
    const url = new URL(request.url);
    const category = url.searchParams.get('category');

    const products = [
      { id: 1, name: 'Running Shoes', price: 2999, category: 'shoes' },
      { id: 2, name: 'Formal Shoes', price: 4999, category: 'shoes' },
      { id: 3, name: 'T-Shirt', price: 799, category: 'clothing' },
    ];

    const filtered = category
      ? products.filter(p => p.category === category)
      : products;

    return HttpResponse.json(filtered);
  }),

  http.post('/api/cart', async ({ request }) => {
    const body = await request.json();
    return HttpResponse.json({ success: true, itemId: body.productId });
  }),
];
```

JAVASCRIPT Copy

```
// mocks/server.js
import { setupServer } from 'msw/node';
import { handlers } from './handlers';

export const server = setupServer(...handlers);
```

JAVASCRIPT Copy

```
// vitest.setup.js
import { beforeAll, afterEach, afterAll } from 'vitest';
```

```
afterEach(() => server.resetHandlers());
afterAll(() => server.close());
```

Integration test — full component with API calls:

JSX Copy

```jsx
// ProductList.test.jsx
import { render, screen, waitFor } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { http, HttpResponse } from 'msw';
import { server } from '../mocks/server';
import ProductList from './ProductList';

describe('ProductList', () => {
  it('loads and displays products', async () => {
    render(<ProductList category="shoes" />);

    expect(screen.getByText(/loading/i)).toBeInTheDocument();

    await waitFor(() => {
      expect(screen.getByText('Running Shoes')).toBeInTheDocument();
      expect(screen.getByText('Formal Shoes')).toBeInTheDocument();
    });

    expect(screen.queryByText('T-Shirt')).not.toBeInTheDocument();
  });

  it('shows error state when API fails', async () => {
    server.use(
      http.get('/api/products', () => {
        return new HttpResponse(null, { status: 500 });
      })
    );

    render(<ProductList category="shoes" />);

    await waitFor(() => {
      expect(screen.getByText(/something went wrong/i)).toBeInTheDocument(
    });
```

```
  });

  it('adds product to cart', async () => {
    const user = userEvent.setup();
    render(<ProductList category="shoes" />);

    await waitFor(() => {
      expect(screen.getByText('Running Shoes')).toBeInTheDocument();
    });

    await user.click(screen.getAllByRole('button', { name: /add to cart/i

    await waitFor(() => {
      expect(screen.getByText(/added to cart/i)).toBeInTheDocument();
    });
  });
});
```

Why MSW over mocking fetch/axios directly:

MSW intercepts at the network level — your component's actual fetch code runs. If you mock `fetch` with `vi.fn()`, you're not testing that your URL construction, headers, and error handling work correctly. MSW tests the full request path except the actual network call.

MSW also works in the browser (for Storybook and development), not just Node.js test environments.

Follow-up: "When do you use integration tests vs unit tests vs E2E?"

Unit tests for pure logic (utility functions, data transformations, algorithms). Integration tests for component behavior (rendering, user interaction, API integration). E2E tests for critical user flows (checkout, signup, payment). Most of your tests should be integration tests — they give the highest confidence-to-maintenance ratio.

Problems

Advanced testing

Frontend System Desi...

LEARN MORE

ARTICLES & DOCS

- **React Testing Library — intro**
  testing-library.com
- **Jest — official docs**
  jestjs.io

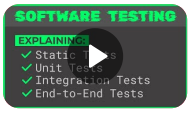
VIDEOS



Unit, integration & e2e tests
YouTube



When to unit, e2e & integration test
YouTube



Static, unit, integration & e2e explained
YouTube

Hand-picked references for this lesson.